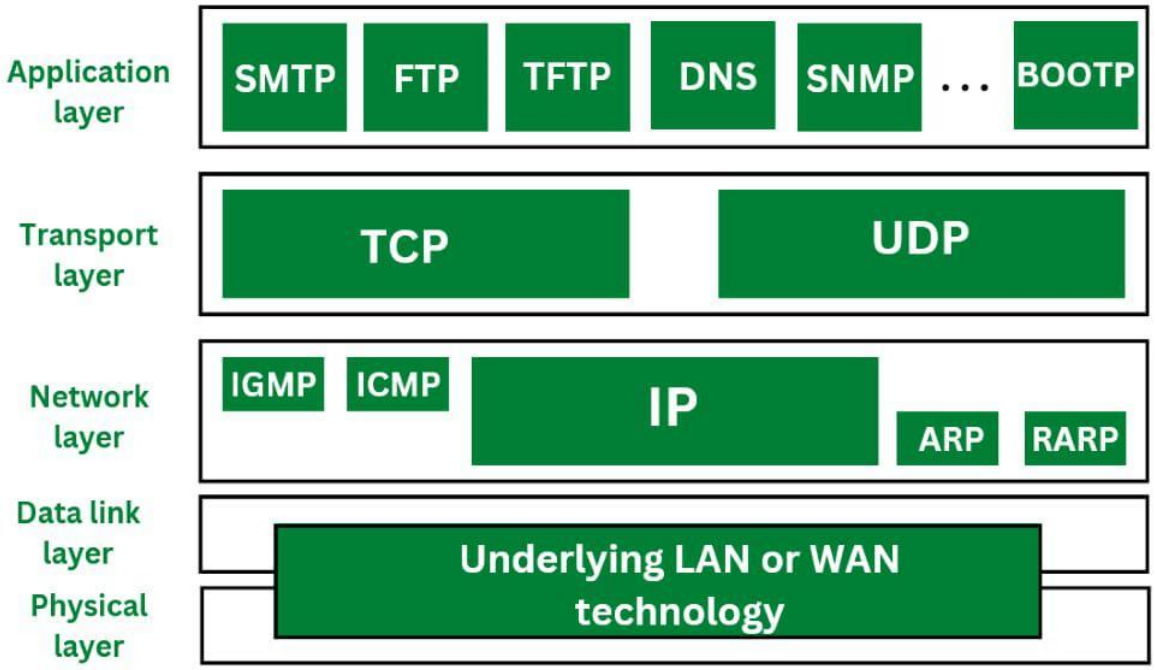
Unit-IV: Transport Layer

❖ Transport Layer Protocols

The transport layer is the fourth layer in the OSI model and the second layer in the TCP/IP model. The transport layer provides with end to end connection between the source and the destination and reliable delivery of the services. Therefore transport layer is known as the end-to-end layer. The transport layer takes the services from its upward layer which is the application layer and provides it to the network layer. Segment is the unit of data encapsulation at the transport layer.

In this article, we are going to discuss all the important aspects of Transport Layer Protocol which include: Functions of Transport Layer protocol, characteristics of TLP, UDP & UDP Segemnts and their Advantages and Disadvantages, TCP & TCP Segemnts and their Advantages and Disadvantages, SCTP and its Advantages & Disadvantages.



Functions of Transport Layer

- The process to process delivery
- End-to-end connection between devices
- Multiplexing and Demultiplexing
- Data integrity and error Correction
- Congestion Control
- Flow Control

Characteristics of Transport Layer Protocol

- The two protocols that make up the transport layer are TCP and UDP.
- A datagram is sent by the IP protocol at the network layer from a source host to a destination host.
- These days, an operating system can support environments with multiple users and processes; a programme under execution is referred to as a process.
- A source process is transmitting a process to a destination process when a host sends a message to another host. Certain connections to certain ports, referred to as protocol ports, are defined by the transport layer protocols.
- A positive integer address, consisting of 16 bits, defines each port.

❖ TCP Connection Establishment

TCP is a connection-oriented protocol and every connection-oriented protocol needs to establish a connection in order to reserve resources at both the communicating ends.

Connection Establishment –

1. Sender starts the process with the following:

- **Sequence number (Seq=521):** contains the random initial sequence number generated at the sender side.
- **Syn flag (Syn=1):** request the receiver to synchronize its sequence number with the above-provided sequence number.
- **Maximum segment size (MSS=1460 B):** sender tells its maximum segment size, so that receiver sends datagram which won't require any fragmentation. MSS field is present inside **Option** field in TCP header.
- **Window size (window=14600 B):** sender tells about his buffer capacity in which he has to store messages from the receiver.

2. TCP is a full-duplex protocol so both sender and receiver require a window for receiving messages from one another.

- **Sequence number (Seq=2000):** contains the random initial sequence number generated at the receiver side.
- **Syn flag (Syn=1):** request the sender to synchronize its sequence number with the above-provided sequence number.
- **Maximum segment size (MSS=500 B):** receiver tells its maximum segment size, so that sender sends datagram which won't require any fragmentation. MSS field is present inside **Option** field in TCP header.
  Since $MSS_{receiver} < MSS_{sender}$, both parties agree for minimum MSS i.e., 500 B to avoid fragmentation of packets at both ends.

Therefore, receiver can send maximum of 14600/500 = 29 packets.

This is the receiver's sending window size.

- **Window size (window=10000 B):** receiver tells about his buffer capacity in which he has to store messages from the sender.

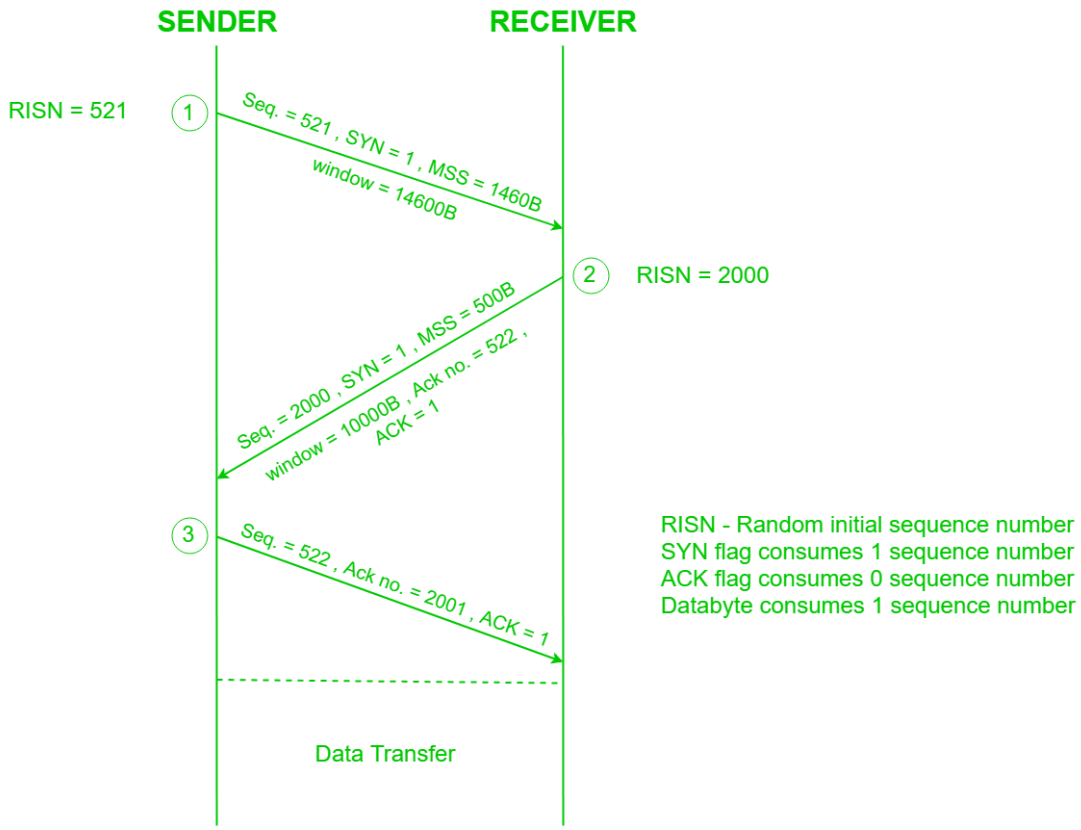
Therefore, sender can send a maximum of 10000/500 = 20 packets.

This is the sender's sending window size.

- **Acknowledgement Number (Ack no.=522):** Since sequence number 521 is received by the receiver so, it makes a request for the next sequence number with Ack no.=522 which is the next packet expected by the receiver since Syn flag consumes 1 sequence no.
- **ACK flag (ACk=1):** tells that the acknowledgement number field contains the next sequence expected by the receiver.

3. Sender makes the final reply for connection establishment in the following way:

- **Sequence number (Seq=522):** since sequence number = 521 in 1st step and SYN flag consumes one sequence number hence, the next sequence number will be 522.
- **Acknowledgement Number (Ack no.=2001):** since the sender is acknowledging SYN=1 packet from the receiver with sequence number 2000 so, the next sequence number expected is 2001.
- **ACK flag (ACK=1):** tells that the acknowledgement number field contains the next sequence expected by the sender.



Since the connection establishment phase of TCP makes use of 3 packets, it is also known as 3-way Handshaking **(SYN, SYN + ACK, ACK).**

❖ **TCP Connection Termination**

In TCP 3-way Handshake Process we studied that how connections are established between client and server in Transmission Control Protocol (TCP) using **SYN** bit segments. In this article, we will study how TCP close connection between Client and Server. Here we will also need to send bit segments to a server which **FIN** bit is set to 1.
TCP supports two types of connection releases like most connection-oriented transport protocols:

1. **Graceful connection release –**
   In the Graceful connection release, the connection is open until both parties have closed their sides of the connection.
2. **Abrupt connection release –**
   In an Abrupt connection release, either one TCP entity is forced to close the connection or one user closes both directions of data transfer.

Abrupt connection release :
An abrupt connection release is carried out when an RST segment is sent. An RST segment can be sent for the below reasons:

1. When a non-SYN segment was received for a non-existing TCP connection.

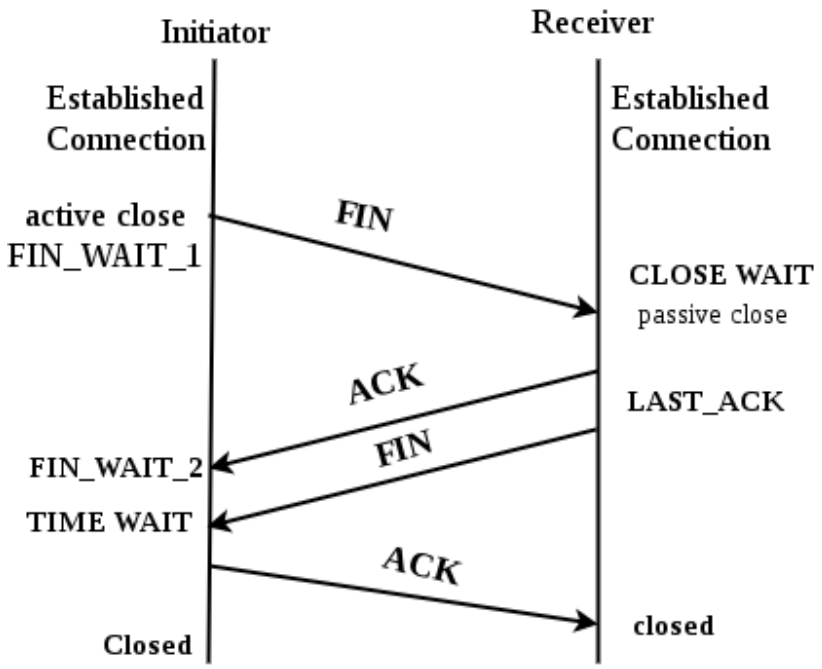
2. In an open connection, some TCP implementations send an RST segment when a segment with an invalid header is received. This will prevent attacks by closing the corresponding connection.

3. When some implementations need to close an existing TCP connection, they send an RST segment. They will close an existing TCP connection for the following reasons:
   - Lack of resources to support the connection

- The remote host is now unreachable and has stopped responding.

When a TCP entity sends an RST segment, it should contain 00 if it does not belong to any existing connection else it should contain the current value of the sequence number for the connection and the acknowledgment number should be set to the next expected in- sequence number on this connection.

Graceful Connection Release :
The common way of terminating a TCP connection is by using the TCP header's FIN flag. This mechanism allows each host to release its own side of the connection individually.



How mechanism works In TCP :

1. **Step 1 (FIN From Client) –**
   Suppose that the client application decides it wants to close the connection. (Note that the server could also choose to close the connection). This causes the client to send a TCP segment with the **FIN** bit set to **1** to the server and to enter the **FIN_WAIT_1** state. While in the **FIN_WAIT_1** state, the client waits for a TCP segment from the server with an acknowledgment (ACK).
2. **Step 2 (ACK From Server) –**
   When the Server received the FIN bit segment from Sender (Client), Server Immediately sends acknowledgement (ACK) segment to the Sender (Client).
3. **Step 3 (Client waiting) –**
   While in the **FIN_WAIT_1** state, the client waits for a TCP segment from the server with an acknowledgment. When it receives this segment, the client enters the **FIN_WAIT_2** state. While in the **FIN_WAIT_2** state, the client waits for another segment from the server with the FIN bit set to 1.
4. **Step 4 (FIN from Server) –**
   The server sends the FIN bit segment to the Sender(Client) after some time when the Server sends the ACK segment (because of some closing process in the Server).
5. **Step 5 (ACK from Client) –**
   When the Client receives the FIN bit segment from the Server, the client acknowledges the server's segment and enters the **TIME_WAIT** state. The **TIME_WAIT** state lets the client resend the final acknowledgment in case the **ACK** is lost. The time spent by clients in

the **TIME_WAIT** state depends on their implementation, but their typical values are 30 seconds, 1 minute, and 2 minutes. After the wait, the connection formally closes and all resources on the client-side (including port numbers and buffer data) are released.

In the below Figures illustrate the series of states visited by the server-side and also the Client-side, assuming the client begins connection tear-down. In these two state-transition figures, we have only shown how a TCP connection is normally established and shut down.

TCP states visited by ClientSide –

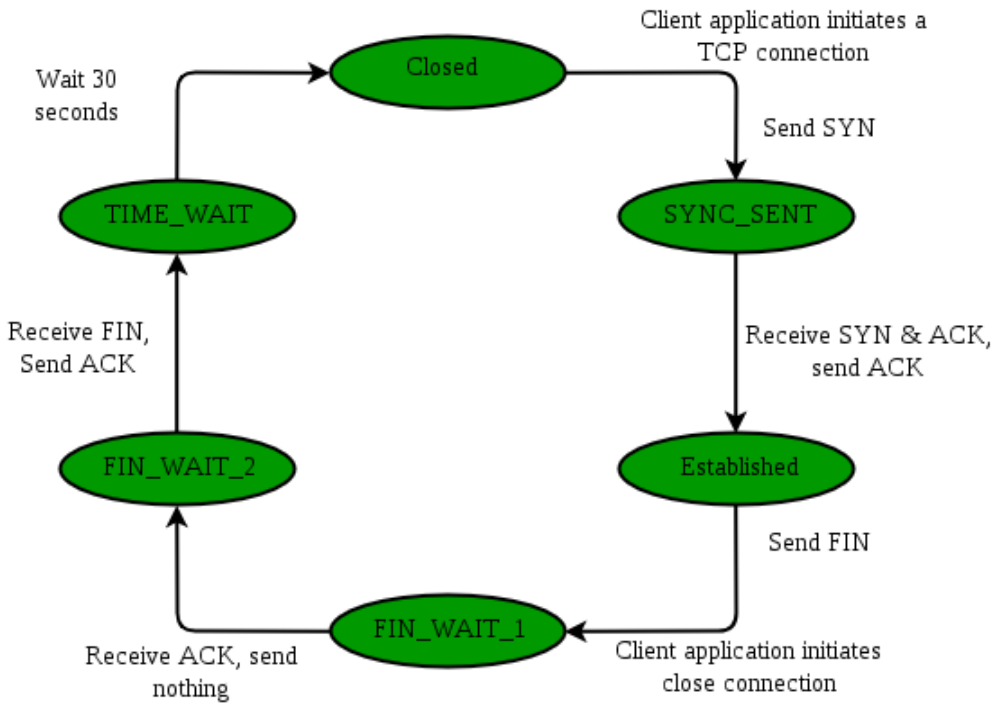


Fig : TCP states visited by a client TCP

TCP states visited by ServerSide –

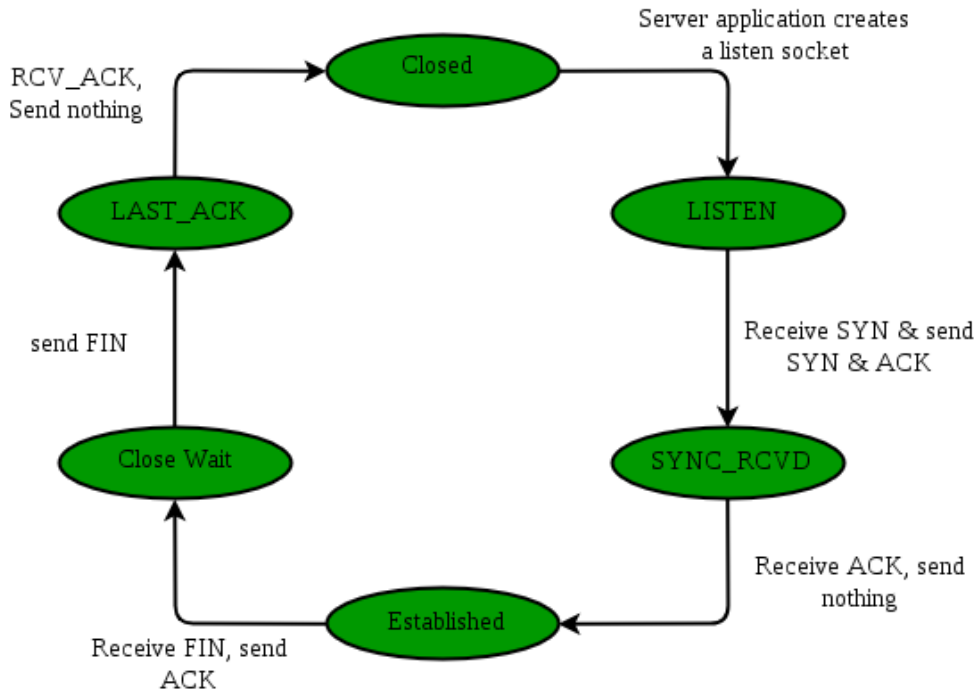


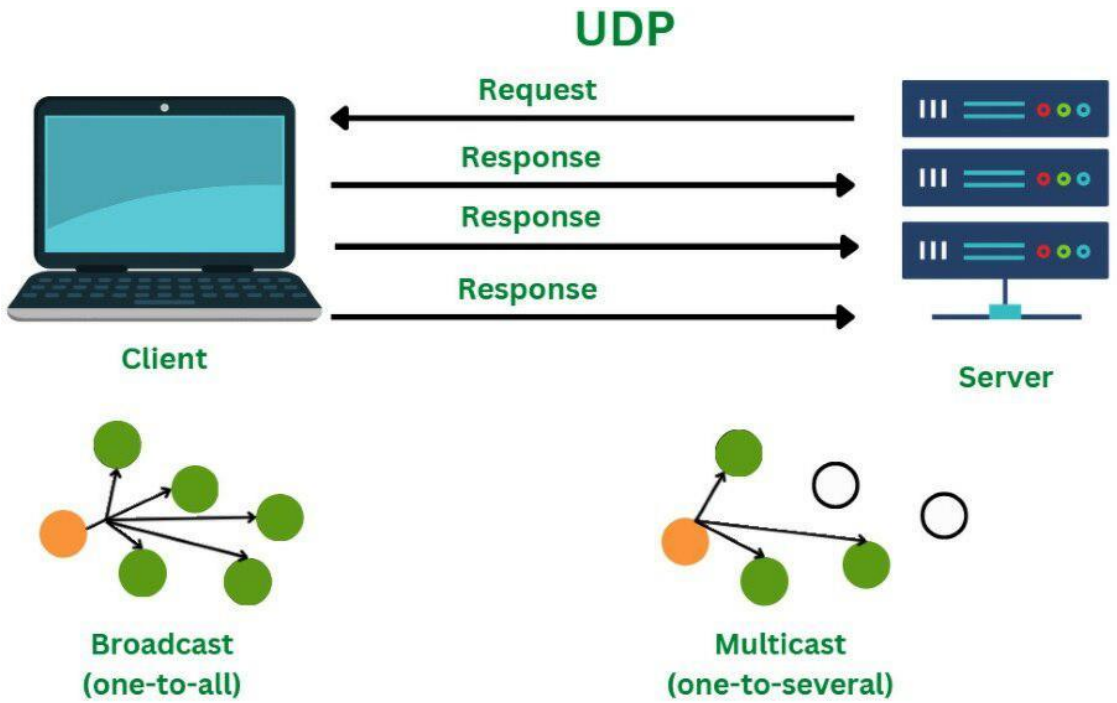
Fig : TCP states visited by a client TCP

❖ Transport Layer Protocols

The transport layer is represented majorly by TCP and UDP protocols. Today almost all operating systems support multiprocessing multi-user environments. This transport layer protocol provides connections to the individual ports. These ports are known as protocol ports. Transport layer protocols work above the IP protocols and deliver the data packets from IP serves to destination port and from the originating port to destination IP services. Below are the protocols used at the transport layer.

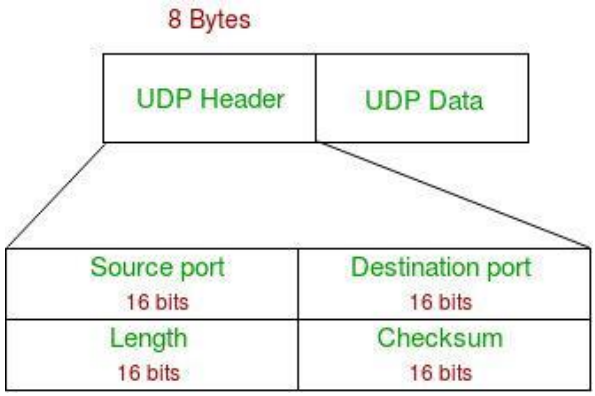
1. UDP

UDP stands for User Datagram Protocol. User Datagram Protocol provides a nonsequential transmission of data. It is a connectionless transport protocol. UDP protocol is used in applications where the speed and size of data transmitted is considered as more important than the security and reliability. User Datagram is defined as a packet produced by User Datagram Protocol. UDP protocol adds checksum error control, transport level addresses, and information of length to the data received from the layer above it. Services provided by User Datagram Protocol(UDP) are connectionless service, faster delivery of messages, checksum, and process-to-process communication.



❖ UDP Segment (UDP Header Format)

While the TCP header can range from 20 to 60 bytes, the UDP header is a fixed, basic 8 bytes. All required header information is contained in the first 8 bytes, with data making up the remaining portion. Because UDP port number fields are 16 bits long, the range of possible port numbers is defined as 0 to 65535, with port 0 being reserved.



UDP

- **Source Port:** Source Port is a 2 Byte long field used to identify the port number of the source.
- **Destination Port:** This 2-byte element is used to specify the packet's destination port.
- **Length:** The whole length of a UDP packet, including the data and header. The field has sixteen bits.
- **Cheksum:** The checksum field is two bytes long. The data is padded with zero octets at the end (if needed) to create a multiple of two octets. It is the 16-bit one's complement of the one's complement sum of the UDP header, the pseudo-header containing information from the IP header, and the data.

Advantages of UDP

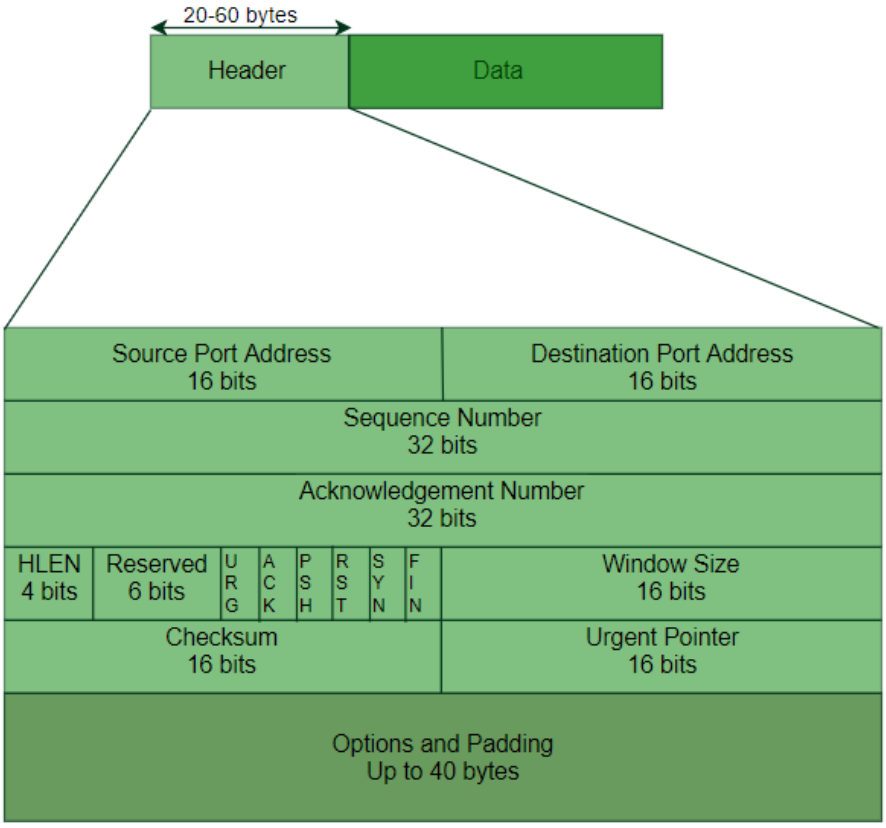
- UDP also provides multicast and broadcast transmission of data.
- UDP protocol is preferred more for small transactions such as DNS lookup.
- It is a connectionless protocol, therefore there is no compulsion to have a connection-oriented network.
- UDP provides fast delivery of messages.

Disadvantages of UDP

- In UDP protocol there is no guarantee that the packet is delivered.
- UDP protocol suffers from worse packet loss.
- UDP protocol has no congestion control mechanism.
- UDP protocol does not provide the sequential transmission of data.

❖ TCP Segment (TCP Header Format)

A TCP segment's header may have 20–60 bytes. The options take about 40 bytes. A header consists of 20 bytes by default, although it can contain up to 60 bytes.



- **Source Port Address:** The port address of the programme sending the data segment is stored in the 16-bit field known as the source port address.
- **Destination Port Address:** The port address of the application running on the host receiving the data segment is stored in the destination port address, a 16-bit field.

- **Sequence Number:** The sequence number, or the byte number of the first byte sent in that specific segment, is stored in a 32-bit field. At the receiving end, it is used to put the message back together once it has been received out of sequence.
- **Acknowledgement Number**: The acknowledgement number, or the byte number that the recipient anticipates receiving next, is stored in a 32-bit field called the acknowledgement number. It serves as a confirmation that the earlier bytes were successfully received.
- **Header Length (HLEN):** This 4-bit field stores the number of 4-byte words in the TCP header, indicating how long the header is. For example, if the header is 20 bytes (the minimum length of the TCP header), this field will store 5 because 5 x 4 = 20, and if the header is 60 bytes (the maximum length), it will store 15 because 15 x 4 = 60. As a result, this field's value is always between 5 and 15.
- **Control flags:** These are six 1-bit control bits that regulate flow control, method of transfer, connection abortion, termination, and establishment. They serve the following purposes:
    - **Urgent:** This pointer is legitimate
    - **ACK:** The acknowledgement number (used in cumulative acknowledgement cases) is valid.
    - **PSH:** Push request
    - **RST:** Restart the link.
    - SYN: Sequence number synchronisation
    - **FIN:** Cut off the communication
    - **Window size:** This parameter provides the sender TCP's window size in bytes.
- **Checksum:** The checksum for error control is stored in this field. Unlike UDP, it is required for TCP.
- **Urgent pointer:** This field is used to point to data that must urgently reach the receiving process as soon as possible. It is only valid if the URG control flag is set. To obtain the byte number of the final urgent byte, the value of this field is appended to the sequence number.

Advantages of TCP

- TCP supports multiple routing protocols.
- TCP protocol operates independently of that of the operating system.
- TCP protocol provides the features of error control and flow control.
- TCP provides a connection-oriented protocol and provides the delivery of data.

Disadvantages of TCP

- TCP protocol cannot be used for broadcast or multicast transmission.
- TCP protocol has no block boundaries.
- No clear separation is being offered by TCP protocol between its interface, services, and protocols.
- In TCP/IP replacement of protocol is difficult.

❖ **Connection Establishment**

1. Sender starts the process with the following:

- **Sequence number (Seq=521):** contains the random initial sequence number generated at the sender side.
- **Syn flag (Syn=1):** request the receiver to synchronize its sequence number with the above-provided sequence number.
- **Maximum segment size (MSS=1460 B):** sender tells its maximum segment size, so that receiver sends datagram which won't require any fragmentation. MSS field is present inside **Option** field in TCP header.
- **Window size (window=14600 B):** sender tells about his buffer capacity in which he has to store messages from the receiver.

2. TCP is a full-duplex protocol so both sender and receiver require a window for receiving messages from one another.

- **Sequence number (Seq=2000):** contains the random initial sequence number generated at the receiver side.
- **Syn flag (Syn=1):** request the sender to synchronize its sequence number with the above-provided sequence number.
- **Maximum segment size (MSS=500 B):** receiver tells its maximum segment size, so that sender sends datagram which won't require any fragmentation. MSS field is present inside **Option** field in TCP header.
  Since $MSS_{receiver} < MSS_{sender}$, both parties agree for minimum MSS i.e., 500 B to avoid fragmentation of packets at both ends.

Therefore, receiver can send maximum of 14600/500 = 29 packets.

This is the receiver's sending window size.

- **Window size (window=10000 B):** receiver tells about his buffer capacity in which he has to store messages from the sender.

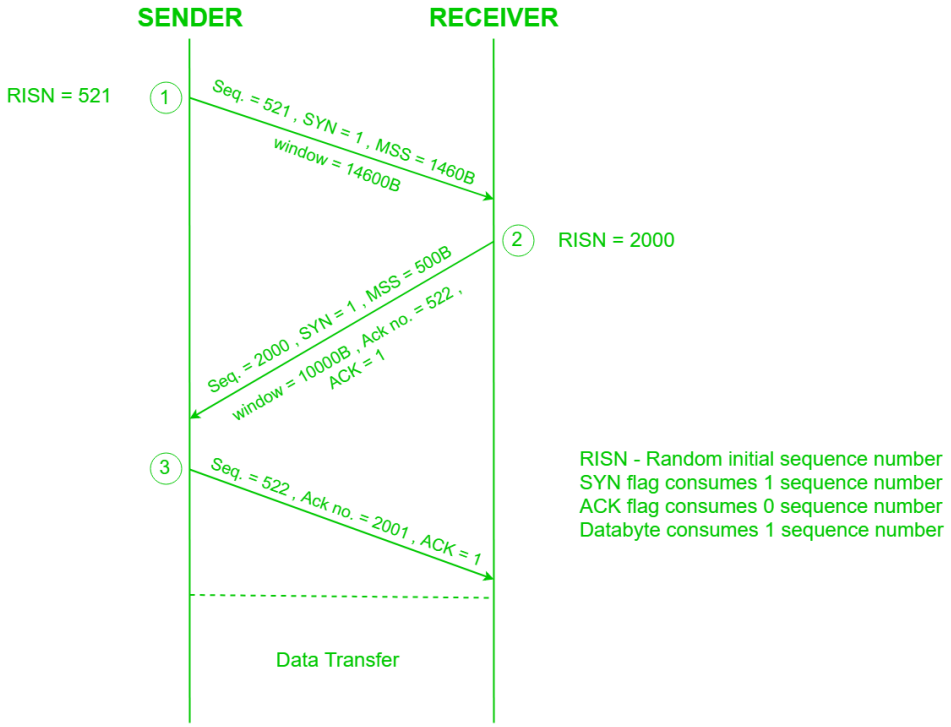
Therefore, sender can send a maximum of 10000/500 = 20 packets.

This is the sender's sending window size.

- **Acknowledgement Number (Ack no.=522):** Since sequence number 521 is received by the receiver so, it makes a request for the next sequence number with Ack no.=522 which is the next packet expected by the receiver since Syn flag consumes 1 sequence no.
- **ACK flag (ACk=1):** tells that the acknowledgement number field contains the next sequence expected by the receiver.

3. Sender makes the final reply for connection establishment in the following way:

- **Sequence number (Seq=522):** since sequence number = 521 in 1st step and SYN flag consumes one sequence number hence, the next sequence number will be 522.
- **Acknowledgement Number (Ack no.=2001):** since the sender is acknowledging SYN=1 packet from the receiver with sequence number 2000 so, the next sequence number expected is 2001.
- **ACK flag (ACK=1):** tells that the acknowledgement number field contains the next sequence expected by the sender.



Since the connection establishment phase of TCP makes use of 3 packets, it is also known as 3-way Handshaking **(SYN, SYN + ACK, ACK).**

❖ Sliding Window Protocol

Reliable data transmission is critical in computer networking, particularly across long distances or in networks that have high latency. The Sliding Window Protocol is a critical component in obtaining this reliability. It is part of the OSI model's Data Link Layer and is used in several protocols, including TCP. Sliding Window protocol handles efficiency issues by sending more than one packet at a time with a larger sequence number.

What is Sliding Window Protocol?

The Sliding Window Protocol is a key computer networking technique for controlling the flow of data between two devices. It guarantees that data is sent consistently and effectively, allowing many packets to be sent before requiring an acknowledgment for the first, maximizing the use of available bandwidth.

Terminologies Related to Sliding Window Protocol

Transmission Delay (Tt) – Time to transmit the packet from the host to the outgoing link. If B is the Bandwidth of the link and D is the Data Size to transmit

Tt = D/B

Propagation Delay (Tp) – It is the time taken by the first bit transferred by the host onto the outgoing link to reach the destination. It depends on the distance d and the wave propagation speed s (depends on the characteristics of the medium).

Tp = d/s

Efficiency – It is defined as the ratio of total useful time to the total cycle time of a packet. For stop and wait protocol,

Total time(TT) = Tt(data) + Tp(data) +
Tt(acknowledgement) + Tp(acknowledgement)
= Tt(data) + Tp(data) + Tp(acknowledgement)
= Tt + 2*Tp

Since acknowledgements are very less in size, their transmission delay can be neglected.

Efficiency = Useful Time / Total Cycle Time
= Tt/(Tt + 2*Tp) (For Stop and Wait)
= 1/(1+2a) [Using a = Tp/Tt]

Effective Bandwidth(EB) or Throughput – Number of bits sent per second.

EB = Data Size(D) / Total Cycle time(Tt + 2*Tp)
Multiplying and dividing by Bandwidth (B),
= (1/(1+2a)) * B [Using a = Tp/Tt]
= Efficiency * Bandwidth

Capacity of link – If a channel is Full Duplex, then bits can be transferred in both the directions and without any collisions. Number of bits a channel/Link can hold at maximum is its capacity.

Capacity = Bandwidth(B) * Propagation(Tp)

For Full Duplex channels,
Capacity = 2*Bandwidth(B) * Propagation(Tp)

Types of Sliding Window Protocol

There are two types of Sliding Window Protocol which include Go-Back-N ARQ and Selective Repeat ARQ:

1. Go-Back-N ARQ

Go-Back-N ARQ allows sending more than one frame before getting the first frame's acknowledgment. It is also known as sliding window protocol since it makes use of the sliding window notion. There is a limit to the amount of frames that can be sent, and they are numbered consecutively. All frames beginning with that frame are retransmitted if the acknowledgment is not received in a timely manner.

2.Selective Repeat ARQ

Additionally, this protocol allows additional frames to be sent before the first frame's acknowledgment is received. But in this case, the excellent frames are received and buffered, and only the incorrect or lost frames are retransmitted.

Advantages of Sliding Window Protocol

- **Efficiency:** The sliding window protocol is an efficient method of transmitting data across a network because it allows multiple packets to be transmitted at the same time. This increases the overall throughput of the network.
- **Reliable:** The protocol ensures reliable delivery of data, by requiring the receiver to acknowledge receipt of each packet before the next packet can be transmitted. This helps to avoid data loss or corruption during transmission.
- **Flexibility:** The sliding window protocol is a flexible technique that can be used with different types of network protocols and topologies, including wireless networks, Ethernet, and IP networks.
- **Congestion Control:** The sliding window protocol can also help control network congestion by adjusting the size of the window based on the network conditions.

Disadvantages of Sliding Window Protocol

- **Complexity:** The sliding window protocol can be complex to implement and can require a lot of memory and processing power to operate efficiently.
- **Delay:** The protocol can introduce a delay in the transmission of data, as each packet must be acknowledged before the next packet can be transmitted. This delay can increase the overall latency of the network.
- **Limited Bandwidth Utilization:** The sliding window protocol may not be able to utilize the full available bandwidth of the network, particularly in high-speed networks, due to the overhead of the protocol.
- **Window Size Limitations:** The maximum size of the sliding window can be limited by the size of the receiver's buffer or the available network resources, which can affect the overall performance of the protocol.

Difference between the Go-Back-N ARQ and Selective Repeat ARQ

Go-Back-N Protocol	Selective Repeat Protocol
In Go-Back-N Protocol, if the sent frame are find suspected then all the frames are re-transmitted from the lost packet to the last packet transmitted.	In selective Repeat protocol, only those frames are re-transmitted which are found suspected.
Sender window size of Go-Back-N Protocol is N.	Sender window size of selective Repeat protocol is also N.
Receiver window size of Go-Back-N Protocol is 1.	Receiver window size of selective Repeat protocol is N.
Go-Back-N Protocol is less complex.	Selective Repeat protocol is more complex.
In Go-Back-N Protocol, neither sender nor at receiver need sorting.	In selective Repeat protocol, receiver side needs sorting to sort the frames.
In Go-Back-N Protocol, type of Acknowledgement is cumulative.	In selective Repeat protocol, type of Acknowledgement is individual.

Go-Back-N Protocol	Selective Repeat Protocol
In Go-Back-N Protocol, Out-of-Order packets are NOT Accepted (discarded) and the entire window is re-transmitted.	In selective Repeat protocol, Out-of-Order packets are Accepted.
In Go-Back-N Protocol, if Receives a corrupt packet, then also, the entire window is re-transmitted.	In selective Repeat protocol, if Receives a corrupt packet, it immediately sends a negative acknowledgement and hence only the selective packet is retransmitted.
Efficiency of Go-Back-N Protocol is N/(1+2*a)	Efficiency of selective Repeat protocol is also N/(1+2*a)

❖ TCP Timers

TCP uses several timers to ensure that excessive delays are not encountered during communications. Several of these timers are elegant, handling problems that are not immediately obvious at first analysis. Each of the timers used by TCP is examined in the following sections, which reveal its role in ensuring data is properly sent from one connection to another.

TCP implementation uses four timers –

- **Retransmission Timer –** To retransmit lost segments, TCP uses retransmission timeout (RTO). When TCP sends a segment the timer starts and stops when the acknowledgment is received. If the timer expires timeout occurs and the segment is retransmitted. RTO (retransmission timeout is for 1 RTT) to calculate retransmission timeout we first need to calculate the RTT(round trip time).
  RTT three types –
    - **Measured RTT(RTTm) –** The measured round-trip time for a segment is the time required for the segment to reach the destination and be acknowledged, although the acknowledgement may include other segments.
    - **Smoothed RTT(RTTs) –** It is the weighted average of RTTm. RTTm is likely to change and its fluctuation is so high that a single measurement cannot be used to calculate RTO.

- **Deviated RTT(RTTd) –** Most implementations do not use RTTs alone so RTT deviated is also calculated to find out RTO.

- **Persistent Timer –** To deal with a zero-window-size deadlock situation, TCP uses a persistence timer. When the sending TCP receives an acknowledgment with a window size of zero, it starts a persistence timer. When the persistence timer goes off, the sending TCP sends a special segment called a probe. This segment contains only 1 byte of new data. It has a sequence number, but its sequence number is never acknowledged; it is even ignored in calculating the sequence number for the rest of the data. The probe causes the receiving TCP to resend the acknowledgment which was lost.
- **Keep Alive Timer –** A keepalive timer is used to prevent a long idle connection between two TCPs. If a client opens a TCP connection to a server transfers some data and becomes silent the client will crash. In this case, the connection remains open forever. So a keepalive timer is used.

Each time the server hears from a client, it resets this timer. The time-out is usually 2 hours. If the server does not hear from the client after 2 hours, it sends a probe segment. If there is no response after 10 probes, each of which is 75 s apart, it assumes that the client is down and terminates the connection.

- **Time Wait Timer –** This timer is used during tcp connection termination. The timer starts after sending the last Ack for 2nd FIN and closing the connection.
  After a TCP connection is closed, it is possible for datagrams that are still making their way through the network to attempt to access the closed port. The quiet timer is intended to prevent the just-closed port from reopening again quickly and receiving these last datagrams.

❖ Session and Presentation Layer

Design issues in Session Layer

Session Layer is one of the Seven Layers of OSI Model. Physical layer, Data Link Layer and Network Layer lack some services such as establishment of a session between communicating systems. This is managed by Session Layer which particularly behaves as a dialog controller between communicating system thus facilitating interaction between them.
Before looking into design issues, here are some of functions of Session Layer:

1. **Dialog Control –**
   Session layer allows two systems to enter into a dialog exchange mechanism which can either be full or half-duplex.

2. **Managing Tokens –**
   The communicating systems in a network try to perform some critical operations and it is Session Layer which prevents collisions which might occur while performing these operations which would otherwise result in a loss.

3. **Synchronization –**
   Checkpoints are the midway marks that are added after a particular interval during stream of data
   transfer. These points are also referred to as synchronization points. The Session layer permits process to add these checkpoints.
   For example, suppose a file of 400 pages is being sent over a network, then it is highly beneficial to set up a checkpoint after every 50 pages so that next 50 pages are sent only when previous pages are received and acknowledged.

Design Issues with Session Layer :

1. **Establish sessions between machines –**
   The establishment of session between machines is an important service provided by session layer. This session is responsible for creating a dialog between connected machines. The Session Layer provides mechanism for opening, closing and managing a session between end-user application processes, i.e. a semi-permanent dialogue. This session consists of requests and responses that occur between applications.

2. **Enhanced Services –**
   Certain services such as checkpoints and management of tokens are the key features of session layer and thus it becomes necessary to keep enhancing these features during the layer's design.

3. **To help in Token management and Synchronization –**
   The session layer plays an important role in preventing collision of several critical operation as well as ensuring better data transfer over network by establishing synchronization points at specific intervals. Thus it becomes highly important to ensure proper execution of these services.

❖ Presentation Layer in OSI model

Introduction: Presentation Layer is the 6th layer in the Open System Interconnection (OSI) model. This layer is also known as Translation layer, as this layer serves as a data translator for the network. The data which this layer receives from the Application Layer is extracted and manipulated here as per the required format to transmit over the network. The main responsibility of this layer is to provide or define the data format and encryption. The presentation layer is also called as Syntax layer since it is responsible for maintaining the proper syntax of the data which it either receives or transmits to other layer(s).

Functions of Presentation Layer:

- Presentation layer format and encrypts data to be sent across the network.
- This layer takes care that the data is sent in such a way that the receiver will understand the information (data) and will be able to use the data efficiently and effectively.
- This layer manages the abstract data structures and allows high-level data structures (example-banking records), which are to be defined or exchanged.
- This layer carries out the encryption at the transmitter and decryption at the receiver.
- This layer carries out data compression to reduce the bandwidth of the data to be transmitted (the primary goal of data compression is to reduce the number of bits which is to be transmitted).
- This layer is responsible for interoperability (ability of computers to exchange and make use of information) between encoding methods as different computers use different encoding methods.
- This layer basically deals with the presentation part of the data.
- Presentation layer, carries out the data compression (number of bits reduction while transmission), which in return improves the data throughput.
- This layer also deals with the issues of string representation.
- The presentation layer is also responsible for integrating all the formats into a standardized format for efficient and effective communication.
- This layer encodes the message from the user-dependent format to the common format and vice-versa for communication between dissimilar systems.
- This layer deals with the syntax and semantics of the messages.
- This layer also ensures that the messages which are to be presented to the upper as well as the lower layer should be standardized as well as in an accurate format too.
- Presentation layer is also responsible for translation, formatting, and delivery of information for processing or display.
- This layer also performs serialization (process of translating a data structure or an object into a format that can be stored or transmitted easily).

Features of Presentation Layer in the OSI model: Presentation layer, plays a vital role while communication is taking place between two devices in a network.
List of features which are provided by the presentation layer are:

- Presentation layer could apply certain sophisticated compression techniques, so fewer bytes of data are required to represent the information when it is sent over the network.
- If two or more devices are communicating over an encrypted connection, then this presentation layer is responsible for adding encryption on the sender’s end as well as the decoding the encryption on the receiver’s end so that it can represent the application layer with unencrypted, readable data.

- This layer formats and encrypts data to be sent over a network, providing freedom from compatibility problems.
- This presentation layer also negotiates the Transfer Syntax.
- This presentation layer is also responsible for compressing data it receives from the application layer before delivering it to the session layer and thus improves the speed as well as the efficiency of communication by minimizing the amount of the data to be transferred.

Working of Presentation Layer in the OSI model:

Presentation layer in the OSI model, as a translator, converts the data sent by the application layer of the transmitting node into an acceptable and compatible data format based on the applicable network protocol and architecture. Upon arrival at the receiving computer, the presentation layer translates data into an acceptable format usable by the application layer. Basically, in other words, this layer takes care of any issues occurring when transmitted data must be viewed in a format different from the original format. Being the functional part of the OSI mode, the presentation layer performs a multitude (large number of) data conversion algorithms and character translation functions. Mainly, this layer is responsible for managing two network characteristics: protocol (set of rules) and architecture.

Presentation Layer Protocols:

Presentation layer being the 6th layer, but the most important layer in the OSI model performs several types of functionalities, which makes sure that data which is being transferred or received should be accurate or clear to all the devices which are there in a closed network. Presentation Layer, for performing translations or other specified functions, needs to use certain protocols which are defined below –

- **Apple Filing Protocol (AFP):** Apple Filing Protocol is the proprietary network protocol (communications protocol) that offers services to macOS or the classic macOS. This is basically the network file control protocol specifically designed for Mac-based platforms.

- **Lightweight Presentation Protocol (LPP):** Lightweight Presentation Protocol is that protocol which is used to provide ISO presentation services on the top of TCP/IP based protocol stacks.

- **NetWare Core Protocol (NCP):** NetWare Core Protocol is the network protocol which is used to access file, print, directory, clock synchronization, messaging, remote command execution and other network service functions.

- **Network Data Representation (NDR):** Network Data Representation is basically the implementation of the presentation layer in the OSI model, which provides or defines various primitive data types, constructed data types and also several types of data representations.

- **External Data Representation (XDR):** External Data Representation (XDR) is the standard for the description and encoding of data. It is useful for transferring data between computer architectures and has been used to communicate data between very diverse machines. Converting from local representation to XDR is called encoding, whereas converting XDR into local representation is called decoding.

- **Secure Socket Layer (SSL):** The Secure Socket Layer protocol provides security to the data that is being transferred between the web browser and the server. SSL encrypts the link between a web server and a browser, which ensures that all data passed between them remains private and free from attacks.

****